音视频同步的特殊情况

之前说的情况中，是使用音频来做视频的同步，现在的问题是如果一旦没有音频，那么同步该怎么办？
鉴于上面的考虑，下面给出解决方案:
方案 1：测两帧图片的时间，让其间隔时间超过正常的间隔时间

```
double fps = av_q2d(is->video_st->r_frame_rate);
double pts_diff = 1/ fps ;
```

可以计算出两帧之间的时差.
然后针对这个时间进行延时就可以了

```
is->cur_pts_time = av_gettime();

double fps = av_q2d(is->video_st->r_frame_rate);
double pts_diff = 1/ fps ;

while( is->last_pts_time != 0 && (is->cur_pts_time - is->last_pts_time < pts_diff*1000000) )
{
        SDL_Delay(1);
        is->cur_pts_time = av_gettime();
}
```

然后在发送完图片之后，读取上一次的时间

```
if( is->audioStream == -1 )
    is->last_pts_time = av_gettime();
```

存在的问题 时间不准

方案 2: 使用 SDL 定时器
如何使用 SDL 定时器
SDL_TimerID SDL_AddTimer(Uint32 interval, SDL_TimerCallback callback, void *param);
参数说明：

- interval：表示定时器的触发间隔，单位为毫秒；
- callback：表示定时器触发时调用的回调函数；
- param：表示传递给回调函数的参数。

返回值说明：

- 返回一个 SDL_TimerID 类型的变量，表示创建的定时器的唯一标识符。如果创建定时器失败，则返回 0。

```
if( m_videoState.audioStream != -1 ){
//创建视频线程
    m_videoState.video_tid = SDL_CreateThread( video_thread , "video_thread" , &m_videoState );

}else{
    double fps = av_q2d(m_videoState.video_st->r_frame_rate);
```

```
    double pts_diff = 1/ fps ;
    //获取画面的间隔时间
    SDL_TimerID timer_id = SDL_AddTimer( pts_diff*1000, timer_callback, &m_videoState);
    if (timer_id == 0) {
        fprintf(stderr, "SDL_AddTimer Error: %s\n", SDL_GetError());
        return;
    }
}

Uint32 timer_callback(Uint32 interval, void *param)
{
    VideoState *is = (VideoState *)param;
    AVPacket pkt1, *packet = &pkt1;
    int ret, got_picture, numBytes;
    double video_pts = 0; //当前视频的 pts
    double audio_pts = 0; //音频 pts
    ///解码视频相关
    AVFrame *pFrame, *pFrameRGB;
    uint8_t *out_buffer_rgb; //解码后的 rgb 数据
    struct SwsContext *img_convert_ctx; //用于解码后的视频格式转换
    AVCodecContext *pCodecCtx = is->pCodecCtx; //视频解码器
    pFrame = av_frame_alloc();
    pFrameRGB = av_frame_alloc();
    ///这里我们改成了 将解码后的 YUV 数据转换成 RGB32
    img_convert_ctx = sws_getContext(pCodecCtx->width, pCodecCtx->height,
                                     pCodecCtx->pix_fmt, pCodecCtx->width, pCodecCtx->height,
                                     AV_PIX_FMT_RGB32, SWS_BICUBIC, NULL, NULL, NULL);
    numBytes = avpicture_get_size(AV_PIX_FMT_RGB32,
                                  pCodecCtx->width,pCodecCtx->height);
    out_buffer_rgb = (uint8_t *) av_malloc(numBytes * sizeof(uint8_t));
    avpicture_fill((AVPicture *) pFrameRGB, out_buffer_rgb, AV_PIX_FMT_RGB32,
                   pCodecCtx->width, pCodecCtx->height);
    do
    {
        if (packet_queue_get(is->videoq, packet, 1) <= 0) break;//队列里面没有数据了
        //读取完毕了
        if( is->audioStream != -1 ){
            while(1)
            {
                audio_pts = is->audio_clock;
                if (video_pts <= audio_pts) break;
                SDL_Delay(5);
```

```
            }
        }

        ret = avcodec_decode_video2(pCodecCtx, pFrame, &got_picture,packet);
        if (ret < 0) {
            av_log(NULL, AV_LOG_ERROR, "Error decoding video frame\n");
            break;
        }
        //获取显示时间 pts
        video_pts = pFrame->pts = pFrame->best_effort_timestamp;
        video_pts *= 1000000 *av_q2d(is->video_st->time_base);
        video_pts = synchronize_video(is, pFrame, video_pts);//视频时钟补偿

        if (got_picture) {
            sws_scale(img_convert_ctx,
                      (uint8_t const * const *) pFrame->data,
                      pFrame->linesize, 0, pCodecCtx->height, pFrameRGB->data,
                      pFrameRGB->linesize);
            //把这个 RGB 数据 用 QImage 加载
            QImage tmpImg((uchar
*)out_buffer_rgb,pCodecCtx->width,pCodecCtx->height,QImage::Format_RGB32);
            QImage image = tmpImg.copy(); //把图像复制一份 传递给界面显示
            is->m_player->SendGetOneImage(image); //调用激发信号的函数

        }
        av_free_packet(packet); //新版考虑使用 av_packet_unref() 函数来代替
    }while(0);

    av_free(pFrame);
    av_free(pFrameRGB);
    av_free(out_buffer_rgb);

    return interval;
}
```

停止时，可以关闭定时器，通过函数 SDL_RemoveTimer(timer_id); 实现.